

ATLAS

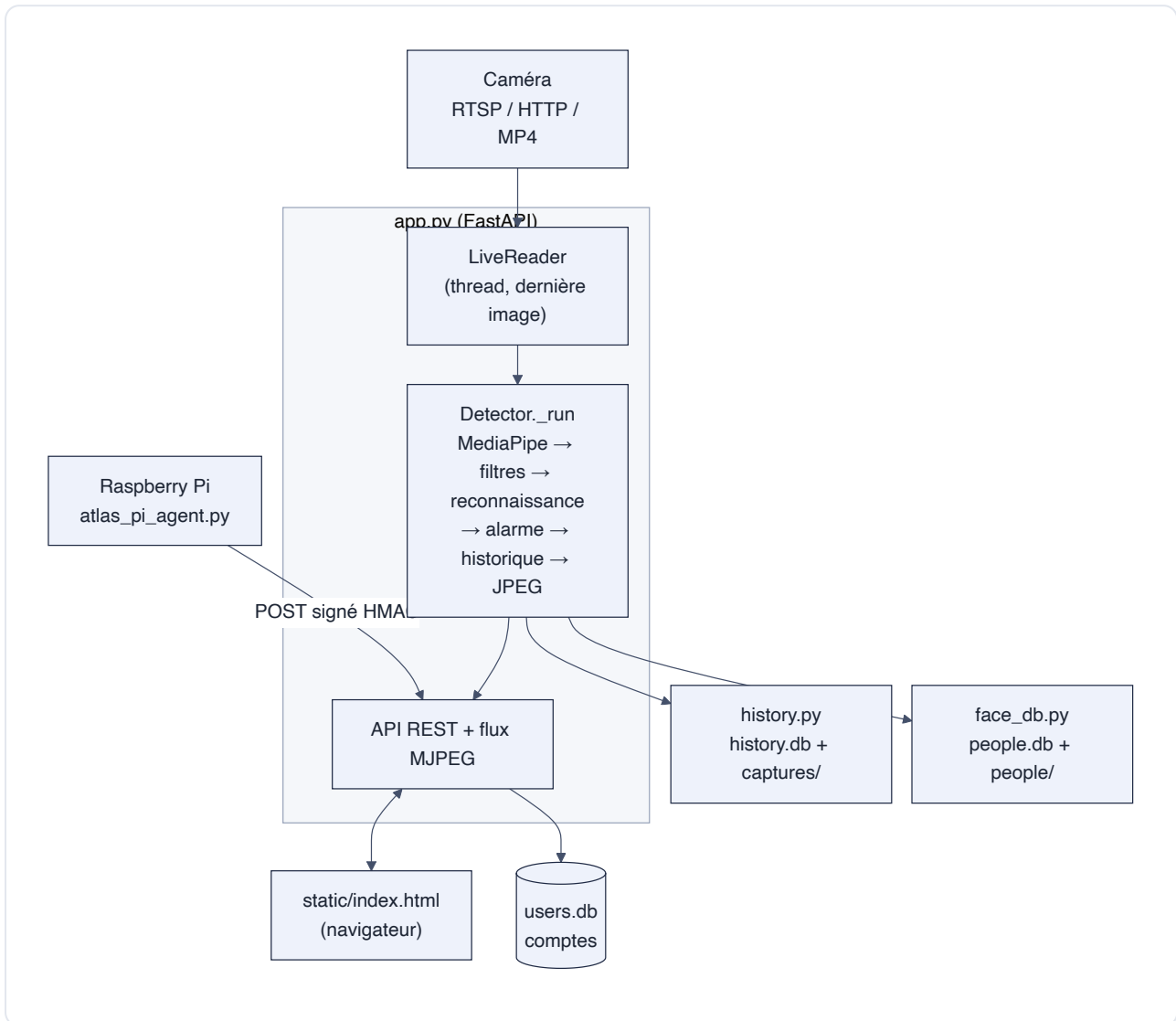
Documentation technique

Détection de visages en temps réel sur un flux vidéo,
reconnaissance des personnes connues, alarme à code et
historique. Backend Python (FastAPI), interface web vanilla JS.

Version du 8 octobre 2026

Détection de visages en temps réel sur un flux vidéo, reconnaissance des personnes connues, alarme à code et historique. Backend Python (FastAPI), interface web vanilla JS.

1. Vue d'ensemble



Fichier	Rôle
<code>app.py</code>	API, authentification, boucle de détection, alarme, notifications, métriques système
<code>face_db.py</code>	Base de visages connus + reconnaissance (embeddings)
<code>history.py</code>	Historique des passages (SQLite + photos)
<code>atlas_pi_agent.py</code>	Agent autonome qui envoie les métriques du Raspberry Pi
<code>seed_demo.py</code>	Crée/synchronise les comptes depuis <code>.env</code>
<code>static/index.html</code>	Interface (SPA)

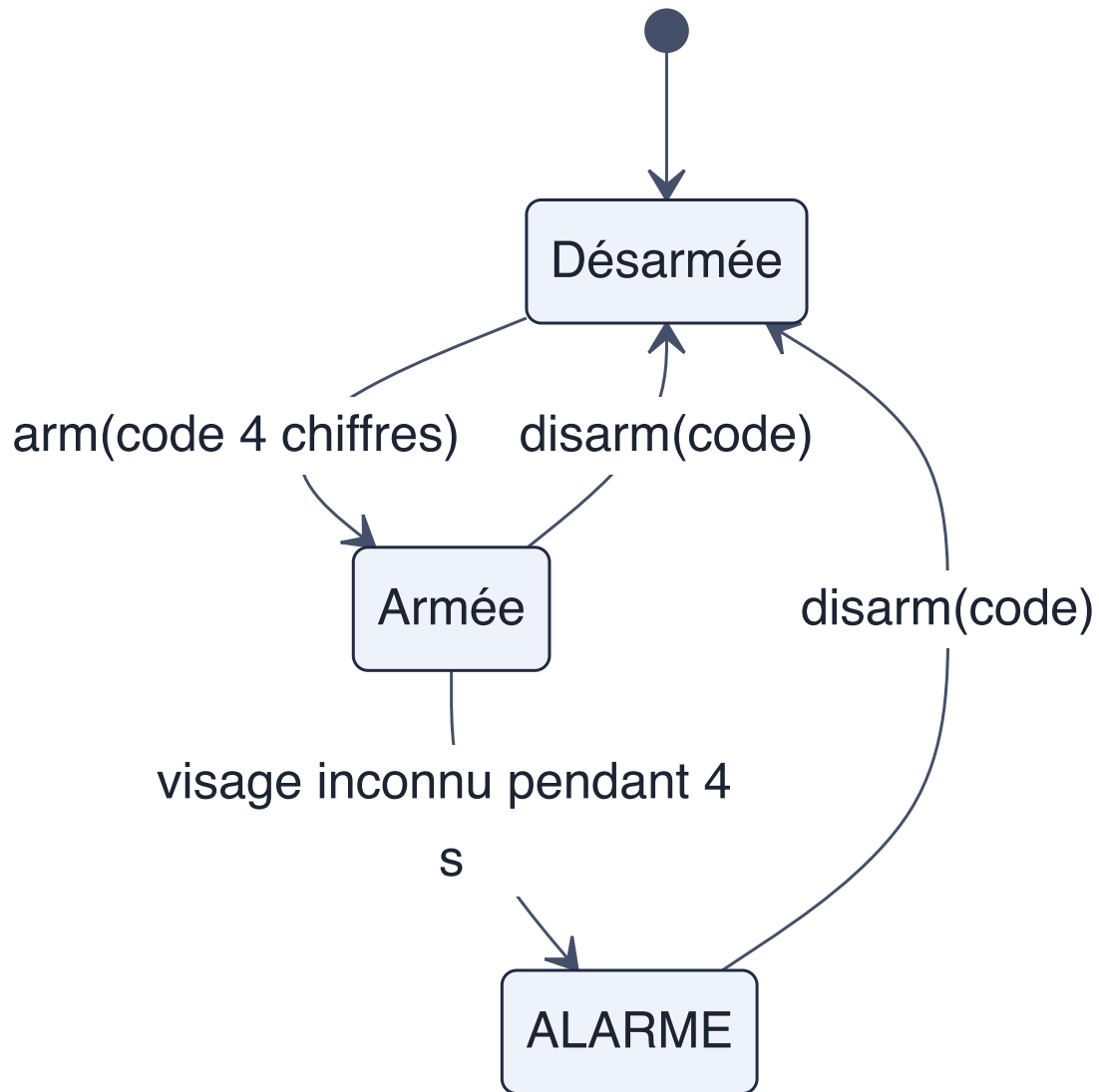
2. Pipeline de détection (`Detector._run`)

Un seul thread d'analyse, lancé par `POST /api/start` ou `/api/upload` .

1. **Ouverture** du flux (`open_capture`) : essaie `CAP_ANY` puis `CAP_FFMPEG` . En cas d'échec ou de coupure, **reconnexion automatique** toutes les 2 s.
2. **Lecture** : pour un flux, `LiveReader` lit en continu dans un thread et ne garde que **la dernière image** (pas de retard accumulé). Pour un fichier, lecture à la cadence normale, avec pause/seek.
3. **Réduction** de l'image à 960 px de large max.
4. **Détection** de visages avec MediaPipe BlazeFace (`models/blaze_face_short_range.tflite` , téléchargé au premier lancement).
5. **Filtres anti-faux positifs** : score ≥ 0.75 , largeur ≥ 4 % de l'image, **3 images consécutives** avec détection.
6. **Reconnaissance** (si `face_db` dispo) : au plus 1 fois / 2 s, sur le meilleur visage uniquement.
7. **Alarme** (voir §3).
8. **Annotation** (cadre rouge « Humain », vert + nom si reconnu), encodage JPEG, publication pour le flux MJPEG.
9. **Historique** (voir §4).

Constantes en tête de `app.py` : `MIN_SCORE` , `MIN_FACE_RATIO` , `CONFIRM_FRAMES` , `MAX_WIDTH` , `HOLD_SECONDS` (1 s, maintien de l'état « détecté »), `EPISODE_GAP` (3 s, fin d'un passage).

3. Alarme



- Le code (4 chiffres) est choisi à l'armement, gardé **en mémoire** uniquement, comparé en temps constant (`hmac.compare_digest`).
- Armée + visage **inconnu** détecté en continu 4 s -> `alarm = True` et notification (`notify`) envoyée dans un thread à part.
- Personne **reconnue** → le compte à rebours est annulé (pas d'alarme).
- Une alarme active ne s'arrête que par `disarm` avec le bon code.
- `GET /api/status` expose `armed` , `alarm` et `countdown` (secondes restantes).
- **Notifications** : `ntfy` (`NTFY_TOPIC`) et/ou email SMTP (`SMTP_HOST` , `ALERT_EMAIL_TO` ...), avec la photo en pièce jointe.

4. Historique (`history.py`)

Un **événement** = un **passage** : créé au premier visage validé, mis à jour tant que la personne est là, clos après 3 s sans visage.

- Table `events` (SQLite) : début/fin, source, nombre de visages, score, alarme active, personne reconnue, boîte du visage, noms des photos.
- Deux photos par passage dans `data/captures/` : image complète + visage rogné. La **meilleure prise** (score le plus haut) remplace la précédente.
- Limite `HISTORY_MAX = 2000` : les plus anciens événements et leurs photos sont purgés.
- Fournit liste filtrée (date, alarmes seules), statistiques (total, aujourd'hui, 24 h, histogramme horaire) et export CSV.
- Un seul `threading.Lock` protège la connexion SQLite partagée.

5. Reconnaissance faciale (`face_db.py`)

- Modèle **InceptionResnetV1** (facenet-pytorch, poids `vggface2`) → embedding de 512 valeurs, chargé à la demande.
- Comparaison par **distance cosinus** ; match si distance ≤ `FACE_RECOGNITION_THRESHOLD` (défaut `0.5` , plus bas = plus strict).
- Embeddings stockés en JSON dans `data/people.db` , mis en cache mémoire (invalidé à chaque ajout/suppression).
- Si `torch` / `facenet-pytorch` sont absents : `available = False` , la détection fonctionne mais personne n'est reconnu.
- Debug : `FACE_RECOGNITION_DEBUG=1` affiche les distances.

6. Authentification et sécurité

- **Comptes** dans `data/users.db` , deux rôles : `admin` et `user` . Créés/synchronisés au démarrage depuis `.env` .
- **Mots de passe** : PBKDF2-SHA256, 200 000 itérations, sel aléatoire.
- **Session** : cookie `argus_session = utilisateur:horodatage:signature` HMAC-SHA256 , `httponly` , `samesite=lax` , durée 24 h (`APP_SESSION_TTL_SECONDS`), `secure` si `APP_SECURE_COOKIES=1` .
- **Anti brute-force** : 5 échecs par couple IP+utilisateur → blocage 5 min (compteur en mémoire).
- **Middlewares** : en-têtes de sécurité (CSP, X-Frame-Options...) ; authentification obligatoire sur `/api/*` , `/` , `/captures` , `/people-photos` (sauf `/login` , `/api/login` , `/api/system/agent` , `/static`).
- **Droits admin** : gérer les personnes, supprimer l'historique. La suppression est de plus **refusée** si **l'alarme est armée**.

7. API

Route	Accès	Description
POST /api/login · /api/logout	public / connecté	Connexion / déconnexion
GET /api/me	connecté	Utilisateur et rôle
POST /api/start {url}	connecté	Démarre l'analyse d'un flux
POST /api/upload	connecté	Envoie et analyse un fichier vidéo
POST /api/stop · pause · resume · seek	connecté	Contrôle de la lecture
GET /api/status	connecté	État (détection, alarme, compte à rebours...)
GET /api/video	connecté	Flux MJPEG annoté
POST /api/alarm/arm · disarm {code}	connecté	Armer / désarmer
GET /api/events	connecté	Historique (limit , offset , alarm , since , until)
GET /api/events/stats · export.csv	connecté	Statistiques / export
DELETE /api/events/{id}	admin	Supprimer l'historique
GET/POST/DELETE /api/people	admin	Gérer les visages connus
GET /api/system/stats	connecté	CPU, RAM, température, disque
POST /api/system/agent	signature HMAC	Réception des métriques du Pi

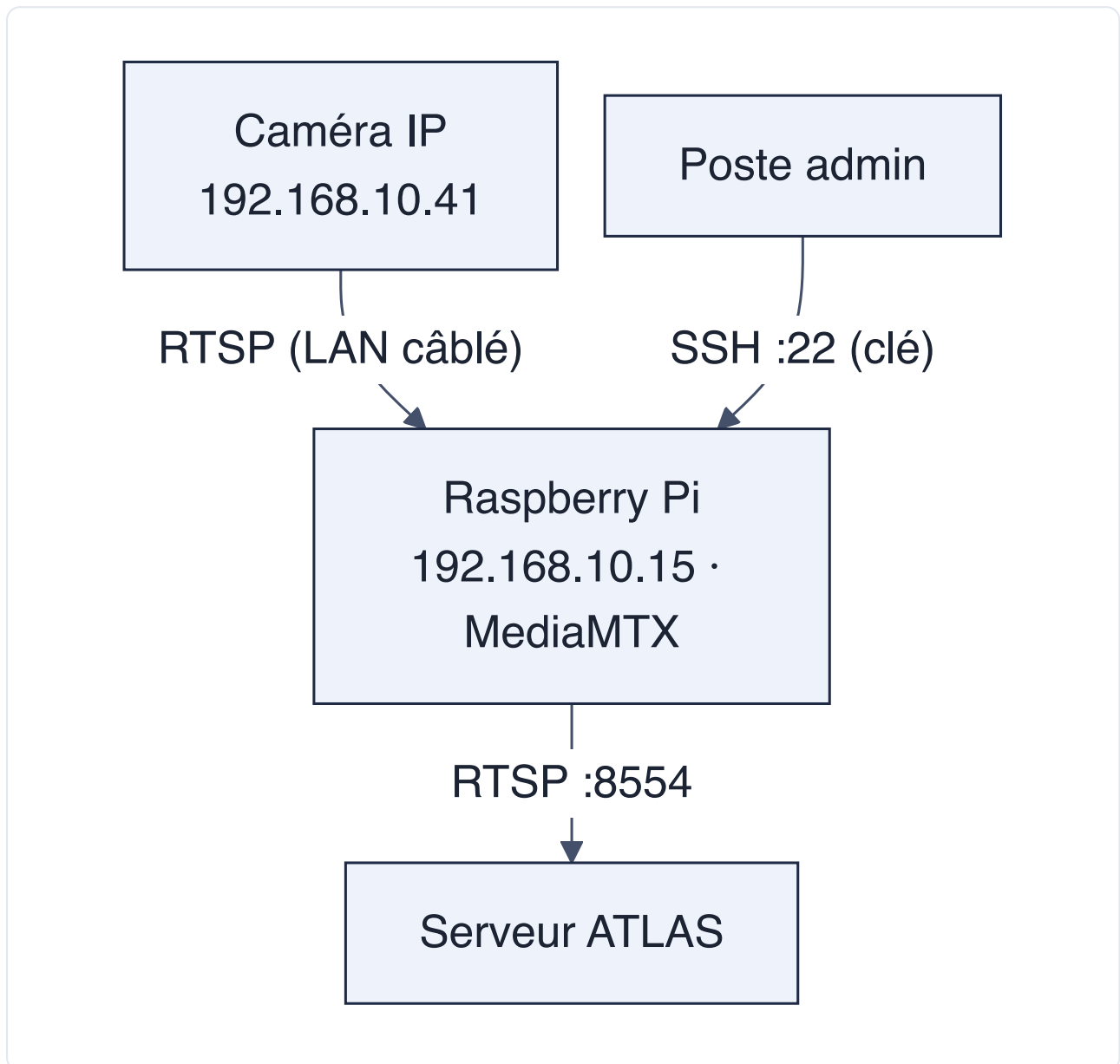
8. Métriques système et agent Raspberry Pi

- GET /api/system/stats renvoie les métriques de l'agent s'il a émis il y a moins de **20 s** (source: raspberry_pi), sinon les métriques **locales** via psutil (source: local).
- **Agent** (atlas_pi_agent.py , stdlib seule) : lit /proc et /sys/class/thermal , envoie un relevé JSON toutes les **5 s**.
- **Signature** : HMAC-SHA256(ATLAS_AGENT_TOKEN, timestamp + "\n" + corps) dans X-Atlas-Signature , avec X-Atlas-Timestamp . Le jeton ne transite jamais.
- **Protections côté serveur** : jeton obligatoire (sinon 503), corps ≤ 4 Ko, horodatage à ± 120 s, validation Pydantic, refus d'un relevé plus ancien que le dernier reçu (rejeu).

- Déploiement du service : voir `README.md` (`systemd`, `/etc/atlas-agent.env`).

9. Infrastructure réseau du Raspberry Pi

Le Pi sert de passerelle entre la caméra IP et ATLAS : il est durci, isolé sur un réseau câblé dédié, et relaie le flux de la caméra. Les commandes exactes ne sont pas reprises ici ; voici ce qui a été fait et pourquoi.



9.1 Système et accès SSH

- Système mis à jour (`apt full-upgrade`).
- **Authentification SSH par clé uniquement** : une clé RSA 4096 bits est générée sur le poste admin et déposée dans `~/.ssh/authorized_keys` du Pi (droits 700/600). Dans `sshd_config` : `PubkeyAuthentication yes` et `PasswordAuthentication no`, ce qui supprime toute attaque par mot de passe sur SSH.

9.2 Pare-feu (UFW)

- Politique par défaut : **tout le trafic entrant refusé**, sortant autorisé.
- Ports ouverts : `22/tcp` (SSH), `443/tcp` (HTTPS), `8883/tcp` (MQTT sur TLS) et `8554/tcp` (RTSP, relais MediaMTX). Tout le trafic entrant sur `eth0` (LAN caméra) est également autorisé.

9.3 Protection anti brute-force (Fail2ban)

- Jail `sshd` configurée dans `/etc/fail2ban/jail.d/sshd.local` (et non en modifiant `jail.conf`) : **3 échecs en 10 min** → **bannissement 1 h**, journaux lus via `systemd`.
- Vérification : `fail2ban-client status sshd` liste les IP bannies.

9.4 Réseau local dédié à la caméra

- Un point d'accès Wi-Fi (`Sentinel-AP` , WPA2-PSK) a été testé puis **abandonné** au profit d'une liaison **Ethernet** directe.
- Le Pi a une IP fixe `192.168.10.15/24` sur `eth0` (profil NetworkManager `Sentinel-LAN`) ; la caméra est sur le même sous-réseau (`192.168.10.41`). La caméra n'est ainsi joignable que depuis le Pi, pas depuis Internet.

9.5 Relais du flux caméra (MediaMTX)

- **MediaMTX** (binaire ARM64) lit le flux RTSP/H.264 de la caméra et le republie sur le chemin `cam` (`rtsp://<ip-du-pi>:8554/cam`). `sourceOnDemand: yes` : la caméra n'est sollicitée que lorsqu'un client est connecté.
- Intérêt : une seule connexion vers la caméra, quel que soit le nombre de lecteurs, et les identifiants de la caméra restent sur le Pi (ils ne figurent que dans `mediamtx.yml` , jamais côté ATLAS).
- Installé comme **service systemd** (`mediamtx.service`) : binaire dans `/usr/local/bin` , config dans `/usr/local/etc/mediamtx.yml` , démarrage automatique, redémarrage après 3 s en cas de crash, lancé sous un utilisateur non-root.
- Supervision : `systemctl status mediamtx` et `journalctl -u mediamtx -f` .
- Dans ATLAS, l'URL à fournir à `POST /api/start` est donc `rtsp://<ip-du-pi>:8554/cam` .

10. Déploiement

- **Local** : `uvicorn app:app --reload` → port 8000.
- **Docker Compose** : `backend` (FastAPI, non publié) + `frontend` (nginx, port **8080**) qui sert `index.html` et proxifie `/api/` et `/captures/` vers le backend. Volumes : `data` , `uploads` , `models` .

11. Données et configuration

Chemin	Contenu
<code>data/users.db</code>	Comptes
<code>data/history.db</code> + <code>data/captures/</code>	Événements + photos
<code>data/people.db</code> + <code>data/people/</code>	Visages connus + photos
<code>uploads/</code>	Vidéos envoyées

Variables `.env` : voir `README.md` (identifiants, `APP_SESSION_SECRET`, `ATLAS_AGENT_TOKEN`, `NTFY_*`, `SMTP_*`, `FACE_RECOGNITION_THRESHOLD`).

12. Limites connues

- **Un seul flux** analysé à la fois ; état (alarme, code, tentatives de login, dernier relevé du Pi) **en mémoire** : perdu au redémarrage, non partagé entre plusieurs workers.
- **Dockerfile.backend** ne copie que `app.py` et `history.py` : `face_db.py` doit y être ajouté pour que l'image démarre.
- Valeurs par défaut des mots de passe et du secret de session à **changer** en production.
- Le détecteur trouve des **visages** ; un humain de dos ou sans visage visible n'est pas détecté.